IIT MADRAS BS DEGREE PROGRAMME

# Computer Systems Design (ESTC)

Digital logic, timing, sequential circuits, FSMs, ISA, ALU and CPU datapaths

Converted from the corresponding Notion study guide. This editable LaTeX source includes typeset equations, tables, recreated TikZ flowcharts, and PGFPlots graphs. The notes are syllabus-aligned study material, not a verbatim transcript of every lecture.

Open original Notion study notes

Compiled 29 October 2026

# Contents

1 Course map and digital-system abstractions 2

2 Abstraction layers and number systems 2

3 Logic gates and Boolean algebra 2

4 Canonical forms and K-maps 3

5 Timing diagrams and delay 3

6 Comparators, multiplexers, encoders and decoders 4

7 Adders and arithmetic circuits 4

8 Sequential logic, registers and counters 4

9 Finite-state machines 5

10 Instruction-set architecture (ISA) 5

11 Arithmetic logic unit (ALU) 5

12 CPU datapath and control integration 6

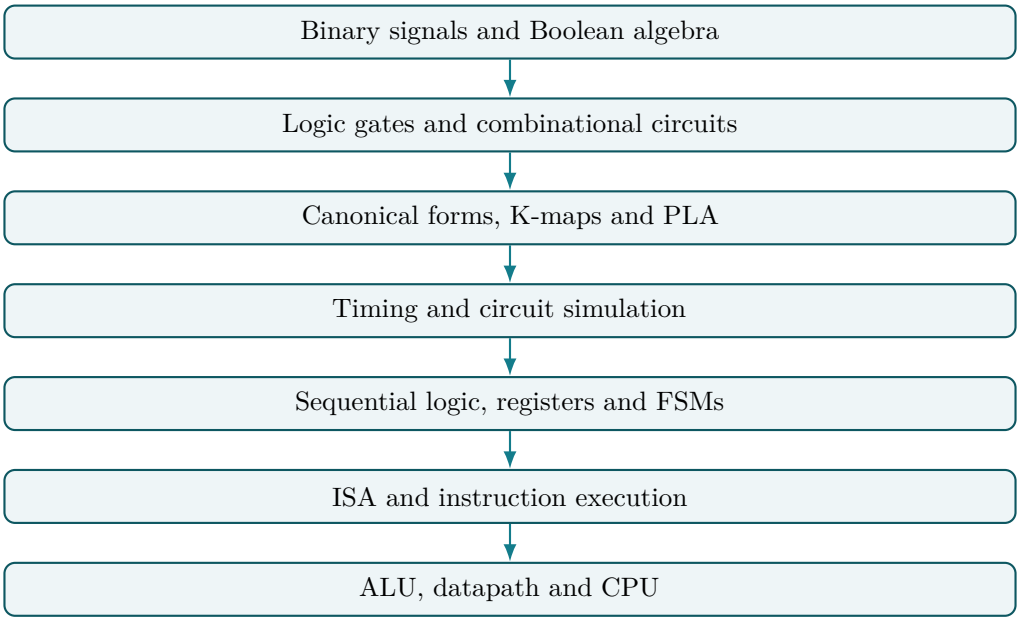
13 Design verification and exam checklist 6

14 Practice questions 6

15 Source and scope 7

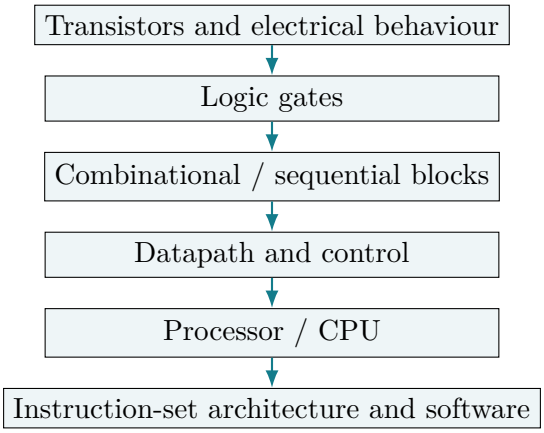
# 1 Course map and digital-system abstractions

Computer Systems Design moves from binary signals and Boolean algebra through combinational and sequential logic to instruction-set architecture, datapaths, the ALU, and CPU integration.




At the logical level a bit is 0 or 1. Combinational outputs depend on current inputs; sequential behaviour also depends on stored state. Real voltage thresholds, noise margins and propagation delays determine how physical signals are interpreted.

# 2 Abstraction layers and number systems




An unsigned binary number has value $\sum_{i=0}^{n-1} b_i 2^i$. Hexadecimal digits represent groups of four bits. Example: $101101_2 = 45_{10} = 2D_{16}$. Two's-complement signed $n$-bit integers range from $-2^{n-1}$ to $2^{n-1} - 1$. Carry out and signed overflow are distinct conditions.

# 3 Logic gates and Boolean algebra

NOT inverts, AND is high only when all inputs are high, OR is high when at least one input is high, and XOR is high when inputs differ. NAND and NOR are functionally complete.

| $A$ | $B$ | AND | OR | XOR | NAND | NOR |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 1 | 1 |
| 0 | 1 | 0 | 1 | 1 | 1 | 0 |
| 1 | 0 | 0 | 1 | 1 | 1 | 0 |
| 1 | 1 | 1 | 1 | 0 | 0 | 0 |

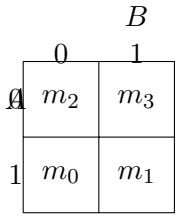
**Core formula**

Identity: $A + 0 = A$, $A \cdot 1 = A$. Domination: $A + 1 = 1$, $A \cdot 0 = 0$. Complement: $A + A' = 1$, $AA' = 0$.
De Morgan: $(AB)' = A' + B'$ and $(A + B)' = A'B'$.

Here + means OR, juxtaposition means AND, and apostrophe means NOT. For $F = AB + AB'$, factor $A$: $F = A(B + B') = A$.

## 4 Canonical forms and K-maps

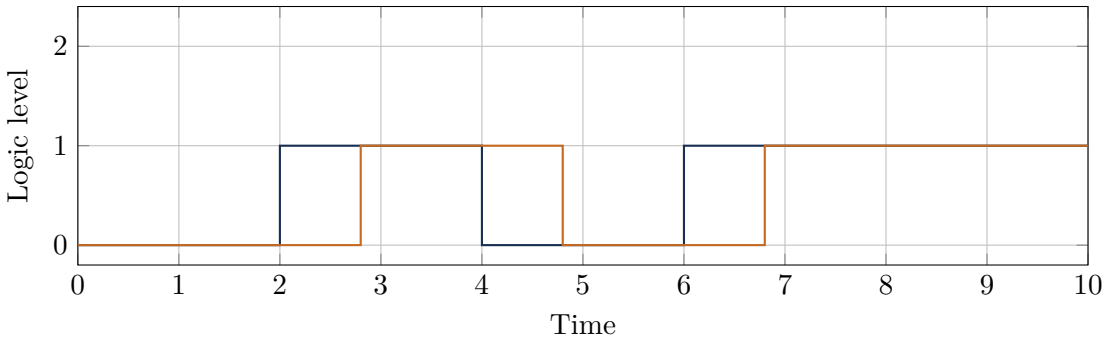
A truth table specifies every output for every input combination. A minterm is an AND term containing each variable once and is 1 on exactly one row. A maxterm is an OR term containing each variable once and is 0 on exactly one row. Canonical SOP ORs minterms for output-1 rows; canonical POS ANDs maxterms for output-0 rows. For $F(A, B) = 1$ on rows 01 and 10, $F = A'B + AB' = A \oplus B$.




K-map cells use Gray-code ordering so adjacent cells differ in one variable. Group required 1-cells in power-of-two rectangles; edge wrapping and overlapping are permitted when they simplify the expression. Do not include a required zero; use don't-cares only when specification allows.

## 5 Timing diagrams and delay

Propagation delay means the time between an input transition and a corresponding output response. Rise/fall time measures transition speed; glitches are short unintended transitions from unequal path delays. Setup and hold specify data stability windows around a storage element's sampling edge.

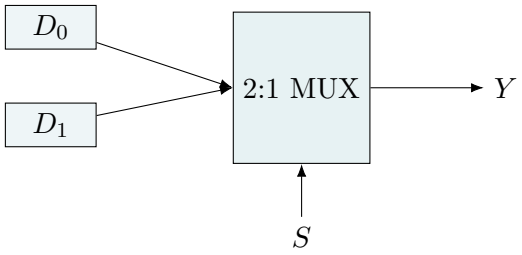


The timing sketch shows an input (navy) and a delayed response (orange). It is conceptual rather than a gate-library timing specification.

## 6 Comparators, multiplexers, encoders and decoders

A one-bit comparator has $A > B = AB'$, $A < B = A'B$, and $A = B = A \,\text{XNOR}\, B$. Multi-bit unsigned comparison begins at the most significant bit; the first differing bit determines the outcome. A 2-to-1 multiplexer selects $D_0$ when $S = 0$ and $D_1$ when $S = 1$:

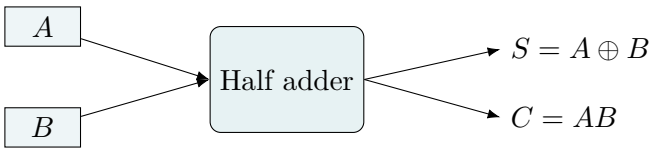
$$Y = S'D_0 + SD_1.$$




A decoder activates one output for an input code; an encoder maps an active input line to a binary code. A simple encoder assumes only one input active; a priority encoder defines behaviour when several are active.

## 7 Adders and arithmetic circuits

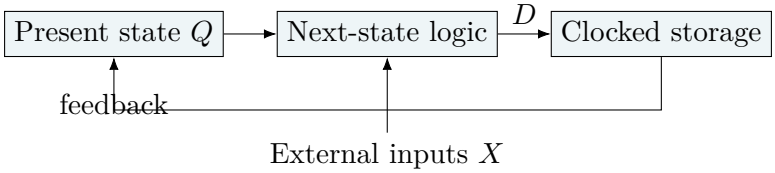
A half adder computes $S = A \oplus B$ and $C = AB$. A full adder includes carry-in: $S = A \oplus B \oplus C_{in}$ and $C_{out} = AB + AC_{in} + BC_{in}$. Ripple carry passes each stage's carry to the next, so worst-case delay tends to rise with width.



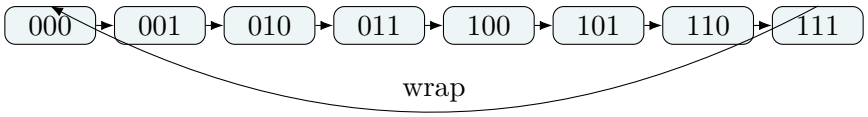

A PLA has a programmable AND plane for product terms and an OR plane for combining those terms; shared products may reduce hardware cost.

## 8 Sequential logic, registers and counters

Combinational logic maps present inputs to outputs. Sequential logic stores state. A latch is level-sensitive; a flip-flop is edge-triggered. For a D flip-flop, $Q_{next} = D$ at the active edge when timing constraints are met. Violating setup or hold can result in incorrect capture or metastability.

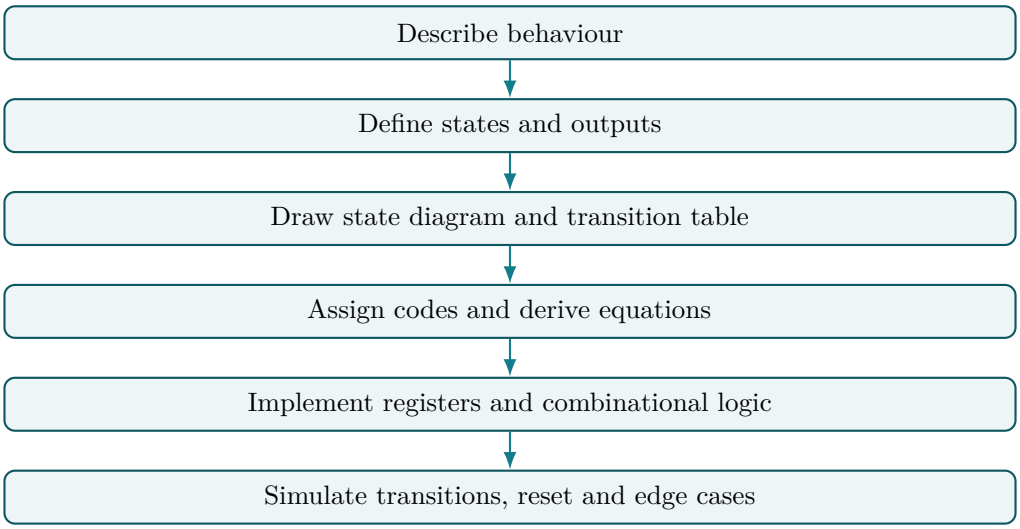


A register is a group of flip-flops storing a word. A binary $n$-bit up-counter cycles from 0 through $2^n - 1$ and wraps under modulo-$2^n$ operation. A modulo-$N$ counter needs at least $\lceil \log_2 N \rceil$ state bits.




## 9 Finite-state machines

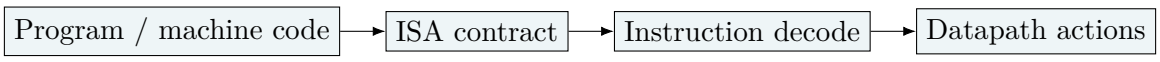
An FSM consists of finite states, inputs, a transition function, and outputs. Moore outputs depend on current state; Mealy outputs depend on current state and current inputs.




Specify reset state, default transitions, priority when inputs coincide, and behaviour for unused states. Missing transitions or ambiguous input priority make a design underspecified.

## 10 Instruction-set architecture (ISA)

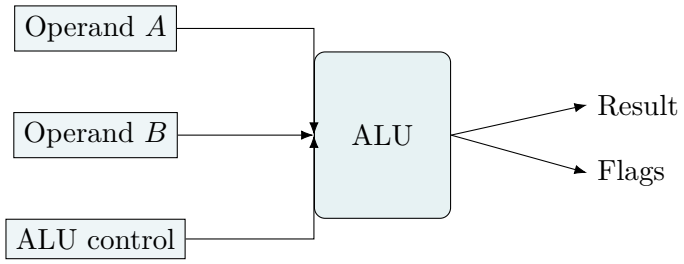
The ISA is the programmer-visible contract: instructions, registers, data types, addressing, memory semantics, exceptions, and control flow. Microarchitecture is the internal design that implements the contract.




Conceptually, instruction execution includes fetch, decode, operand read, execute, optional memory access, write-back, and PC update. Single-cycle, multi-cycle, and pipelined implementations organise this differently.

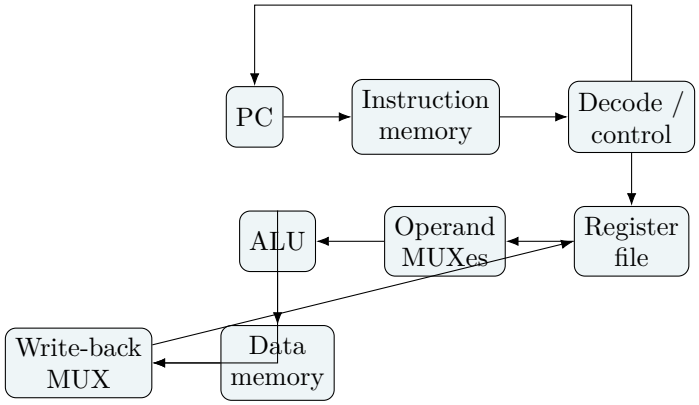
## 11 Arithmetic logic unit (ALU)

An ALU performs addition, subtraction, Boolean operations, shifts, and comparisons. Two's-complement subtraction is often implemented as $A - B = A + \overline{B} + 1$. Carry is primarily an unsigned arithmetic condition; signed overflow indicates that the signed mathematical result exceeds the representable range. Status flags and their definitions depend on the ISA.




## 12 CPU datapath and control integration

A basic CPU integrates a program counter and next-PC logic, instruction memory, decoder/control unit, register file, operand multiplexers, ALU, data-memory interface, and write-back selection. Control signals specify register writes, ALU operation, operand selection, memory accesses, write-back source, and PC choice.




Single-cycle CPUs complete an instruction in one long cycle; multi-cycle designs reuse hardware across shorter cycles; pipelined CPUs overlap instructions across stages but require hazard handling and more complex control.

## 13 Design verification and exam checklist

- Build a complete truth table or transition table.
- Mark active-low signals explicitly and preserve select-bit order.
- Verify K-map adjacency and every required output-one cell.
- Check propagation delay, setup, hold, reset and state transitions.
- Define opcode behaviour and verify ALU flags independently.
- Trace PC, source operands, ALU result, memory effect, and destination write for each instruction.
- Test edge cases and undefined/unused state behaviour against the spec.

## 14 Practice questions

1. Simplify $AB + AB'$. **Answer:** $A(B + B') = A$.
2. When is $A \oplus B = 1$? **Answer:** Inputs differ (01 or 10).
3. For $Y = S'D_0 + SD_1$, what is $Y$ when $S = 1$? **Answer:** $D_1$.

4. Half adder when $A = B = 1$? **Answer:** Sum 0, carry 1.
5. Minimum bits for 10 states? **Answer:** $\lceil \log_2 10 \rceil = 4$.
6. Moore versus Mealy? **Answer:** Moore output depends on state; Mealy output depends on state and input.
7. ISA versus microarchitecture? **Answer:** Contract versus its internal implementation.

## 15 Source and scope

Source: Computer Systems Design – Complete Study Notes and Visual Revision Guide; official outline: `https://study.iitm.ac.in/ds/course_pages/BSCS3031.html`. This note set uses general digital-logic and computer-architecture models, not a specification of a particular CPU or ISA. Confirm course-specific simulator conventions and assessment requirements in the live course portal.